

Clase 4: Bases de Datos y CRUD

En esta clase aprenderemos a darle "memoria" a nuestra aplicación. Pasaremos de manejar datos temporales a guardarlos permanentemente en una base de datos **SQLite** usando **SQLAlchemy**. Para poder ver la base de datos es necesario instalar en VSC **SQLite3 Editor**.

¿Qué es una Base de Datos Relacional?

Imagina una hoja de Excel muy inteligente. La base de datos guarda la información en **Tablas** (filas y columnas).

- Usaremos **SQLite** para trabajar hoy (es un archivo que vive en nuestra carpeta).
- Usaremos **SQLAlchemy**, que es un "traductor". Nos permite crear tablas usando clases de Python. ¡No necesitas aprender comandos de SQL difíciles!

1. Configuración Inicial (El Entorno)

Antes de empezar, asegúrate de que tu entorno virtual esté activo y de tener instalada la librería necesaria:

```
pip install flask-sqlalchemy
```

Y en [main.py](#) colocar:

```
from flask import Flask, render_template, request, redirect, url_for
from flask_sqlalchemy import SQLAlchemy
```

El Concepto CRUD

Cualquier sistema profesional en el mundo (Instagram, Amazon, WhatsApp) hace 4 cosas básicas con los datos:

- **Create (Crear):** Guardar un nuevo producto.
- **Read (Leer):** Mostrar la lista de productos en la web.
- **Update (Actualizar):** Cambiar el precio o nombre de un producto.
- **Delete (Borrar):** Eliminar un producto del catálogo.

Por ello debemos configurar la base de datos de la siguiente forma también desde el `main.py`

```
# 'sqlite:///catalogo.db' creará un archivo real en la carpeta de tu proyecto.  
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///catalogo.db'  
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

```
# Inicializamos la herramienta de Base de Datos  
db = SQLAlchemy(app)
```

2. Definición del Modelo (El "Plano" de la Tabla)

En lugar de crear tablas manualmente, definimos una **clase de Python**.

- **ID:** Es la "llave primaria". Python le asignará un número único a cada producto automáticamente.
- **Columnas:** Definimos qué tipo de dato guardaremos (Texto para el nombre, Float para el precio).

Explicación para alumnos: Imagina que el Modelo es el encabezado de una tabla de Excel. Define qué columnas tendrá nuestro archivo.

Crearemos una clase de acceso para el trabajo con datos para la creación de productos

```
# 3. MODELO DE DATOS (La estructura de nuestra tabla)
```

```
class Producto(db.Model):
```

```
    id = db.Column(db.Integer, primary_key=True) # Identificador único
```

```
    nombre = db.Column(db.String(100), nullable=False) # Texto de hasta 100 caracteres
```

```
    precio = db.Column(db.Float, nullable=False) # Números con decimales
```

```
    def __repr__(self):
```

```
        return f'<Producto {self.nombre}>'
```

```
# 4. CREACIÓN AUTOMÁTICA DE LA BASE DE DATOS
```

```
# Este bloque revisa si el archivo .db existe, si no, lo crea basado en el modelo arriba.
```

```
with app.app_context():
```

```
    db.create_all()
```

Guía de Montaje: Paso a Paso

Paso 1: Definir el "Modelo"

En Python, creamos una clase que represente nuestra tabla. Por ejemplo, la clase `Producto` tendrá:

1. Un id (su número de identificación único).
2. Un nombre.
3. Un precio.

Paso 2: La Primera Migración

La "Migración" es el proceso de transformar nuestras clases de Python en tablas reales dentro del archivo de la base de datos. Es como darle al botón de "construir" el edificio que diseñamos en los planos.

Paso 3: Crear y Leer (El Catálogo)

Aprenderemos a recibir los datos del formulario (como vimos en la Clase 3) y, en lugar de solo mostrarlos en un mensaje, los guardaremos permanentemente en el disco duro.

Paso 4: Actualizar y Borrar

Añadiremos botones de "Editar" y "Eliminar" a nuestro diseño. Cada botón enviará el id del producto a Python para saber exactamente cuál debe modificar o borrar.

3. El Proceso de Migración (`db.create_all`)

Este es el paso técnico más importante. Al ejecutar este comando, Flask revisa tus clases de Python y crea el archivo físico de la base de datos (`catalogo.db`).

- **Nota:** Si cambias la estructura de tu modelo más tarde, deberás borrar el archivo `.db` y volver a ejecutar la aplicación para que se cree con los nuevos cambios.

```
# --- RUTAS DE LA APLICACIÓN (CRUD) ---
```

```
# [R] READ: Listar productos
```

```
@app.route('/')
```

```
def index():
```

```
    # Consultamos TODOS los registros de la tabla Producto
```

```
    productos_db = Producto.query.all()
```

```
    return render_template('index.html', productos=productos_db)
```

```
# [C] CREATE: Agregar un nuevo producto
```

```
@app.route('/agregar', methods=['POST'])
```

```

def agregar_producto():
    # Recibimos los datos del formulario (visto en Clase 3)
    nombre = request.form.get('nombre')
    precio = request.form.get('precio')

    if nombre and precio:
        # Creamos el objeto basado en el Modelo
        nuevo_producto = Producto(nombre=nombre, precio=float(precio))

        # Guardamos en la sesión y confirmamos (commit)
        db.session.add(nuevo_producto)
        db.session.commit()

    # Redireccionamos al inicio para ver el nuevo producto en la lista
    return redirect(url_for('index'))

# [D] DELETE: Borrar un producto por su ID
# Usamos una variable en la URL <int:id> para saber exactamente qué borrar
@app.route('/borrar/<int:id>')
def eliminar_producto(id):
    # Buscamos el producto por su ID único
    producto = Producto.query.get(id)

    if producto:
        db.session.delete(producto)
        db.session.commit()

    return redirect(url_for('index'))

# [U] UPDATE: Editar un producto (Lógica básica)
@app.route('/editar/<int:id>', methods=['POST'])
def editar_producto(id):
    producto = Producto.query.get(id)

    if producto:
        producto.nombre = request.form.get('nombre')
        producto.precio = float(request.form.get('precio'))
        db.session.commit()

    return redirect(url_for('index'))

```

4. Operaciones CRUD (El flujo de trabajo)

[C] Create - Crear

Cuando el usuario envía el formulario:

4. Recibimos los datos con request.form.
5. Creamos un objeto de la clase Producto.
6. Usamos db.session.add() para ponerlo en "espera".

7. Usamos `db.session.commit()` para **guardar definitivamente**.

[R] Read - Leer

Para mostrar los productos en la web, usamos `Producto.query.all()`. Esto trae todos los registros de la base de datos como una lista de objetos que Jinja2 puede dibujar.

[D] Delete - Borrar

Para borrar, necesitamos el **ID**. La URL se ve así: `/borrar/5`. Python busca el producto con el ID 5, lo elimina de la se

```
<h1 class="text-4xl font-extrabold text-gray-800 mb-8">Catálogo de Productos</h1>

<!-- FORMULARIO DE CREACIÓN (C) -->

<section class="bg-indigo-50 p-6 rounded-2xl shadow-sm mb-10">

  <h2 class="text-lg font-bold text-indigo-900 mb-4">Registrar Nuevo Producto</h2>

  <form action="/agregar" method="POST" class="flex flex-wrap gap-4">

    <input type="text" name="nombre" placeholder="Nombre del producto" required

      class="flex-grow p-3 rounded-xl border-none ring-1 ring-gray-300 focus:ring-2
focus:ring-indigo-500 outline-none">

    <input type="number" step="0.01" name="precio" placeholder="Precio ($)" required

      class="w-32 p-3 rounded-xl border-none ring-1 ring-gray-300 focus:ring-2
focus:ring-indigo-500 outline-none">

    <button type="submit" class="bg-indigo-600 text-white px-8 py-3 rounded-xl font-bold
hover:bg-indigo-700 transition">

      Añadir al Inventario

    </button>

  </form>

</section>

<!-- TABLA DE LECTURA (R) Y ACCIONES (U / D) -->

<div class="bg-white rounded-3xl shadow-xl overflow-hidden">
```

```

<table class="w-full text-left">
  <thead class="bg-gray-800 text-white">
    <tr>
      <th class="p-4">ID</th>
      <th class="p-4">Producto</th>
      <th class="p-4">Precio Unitario</th>
      <th class="p-4 text-center">Acciones</th>
    </tr>
  </thead>
  <tbody class="divide-y divide-gray-100">
    {% for p in productos %}
      <tr class="hover:bg-gray-50 transition">
        <td class="p-4 text-gray-400 font-mono">#{{ p.id }}</td>
        <td class="p-4 font-semibold text-gray-700">{{ p.nombre }}</td>
        <td class="p-4 text-indigo-600 font-bold">${{ "%.2f"|format(p.precio) }}</td>
        <td class="p-4 text-center space-x-2">
          <!-- Botón para Borrar (D) -->
          <a href="/borrar/{{ p.id }}"
            class="inline-block bg-red-100 text-red-600 px-3 py-1 rounded-lg text-sm font-bold
            hover:bg-red-200"
            onclick="return confirm('¿Seguro que quieres eliminar este producto?')">
            Eliminar
          </a>
        </td>
      </tr>
    </tbody>
    {% else %}
      <tr>
        <td colspan="4" class="p-10 text-center text-gray-400">
          No hay productos en el catálogo. ¡Agrega el primero arriba!
        </td>
      </tr>
    {% endfor %}
  </tbody>
</table>

```

```
</tr>

{% endfor %}

</tbody>

</table>

</div>
```

sión y confirma el cambio con un commit.

Errores Comunes y Cómo Solucionarlos

1. **Error: "No such table"**
 - **Causa:** No se ejecutó correctamente el comando `db.create_all()`.
 - **Solución:** Revisa que el bloque `with app.app_context():` esté antes de que inicie el servidor.
2. **Los datos no se guardan tras reiniciar**
 - **Causa:** Olvidaste escribir `db.session.commit()`.
 - **Solución:** Sin el "commit", los cambios solo viven en la memoria temporal, no en el disco.
3. **Error de tipos (String vs Float)**
 - **Causa:** Los datos del formulario llegan como texto.
 - **Solución:** Siempre convierte el precio a número usando `float(precio)` antes de guardarlo.

Actividad Sugerida

1. Agreguen 3 productos diferentes.
2. Cerrar la terminal y detener el servidor.
3. Volver a encenderlo.
4. Verifiquen que los 3 productos **siguen ahí**. ¡Felicidades, han logrado la persistencia de datos!